

Agile Demo Project — Complete Beginner Setup Guide

Welcome to the **Agile CI/CD Demo Project!**

This guide walks you **step by step** — from setting up your computer to running and testing your own Agile project connected to GitHub.

No programming or Git knowledge required — just follow carefully and you'll be fine



1. Install Required Tools

You only need to do this once.

1.1 Install Visual Studio Code (VS Code)

Download and install:

<https://code.visualstudio.com/>

During install:

- Check “Add to PATH”
- Allow context menu integration
- Keep all defaults otherwise

1.2 Install Git for Windows

Download:

<https://git-scm.com/download/win>

During setup:

- Choose “**Use Git from the command line and also from 3rd-party software**”
- Keep defaults and finish.

1.3 Install WSL 2 (Linux Subsystem)

Open Windows **PowerShell (Admin)** and run:

```
wsl --install
```

This installs Ubuntu Linux inside Windows.

Restart your computer when asked.

After restart:

- A window called “Ubuntu” will open.
- Set your **username** and **password** (any name, not your Windows one).

You now have Linux on your computer! 🐧

2. Connect VS Code to WSL

1. Open **VS Code**
2. Press **Ctrl + Shift + P**
3. Type and select: WSL: Connect to WSL
4. When it opens, the bottom-left corner of VS Code should show:
5. WSL: Ubuntu

You are now developing *inside Linux* even though you're on Windows.

3. Install VS Code Extensions

In VS Code's left sidebar, click the **Extensions** icon (four squares) and install:

Extension	Purpose
Python	Run and test Python code
GitHub Pull Requests and Issues	Work with GitHub easily
Docker	(optional) For container demos
YAML	Nice syntax for GitHub workflows

4. Set Up Python Environment

In VS Code Terminal (it should say bash or zsh, not PS):

```
sudo apt update
```

```
sudo apt install python3 python3-pip -y
```

```
python3 --version
```

```
pip install --upgrade pip
```

✅ You now have Python.

5. Create the Project Folder

In the same VS Code terminal:

```
cd ~
```

```
mkdir agile-demo
```

```
cd agile-demo
```

6. Initialize Git

```
git init
```

Then tell Git who you are (once only):

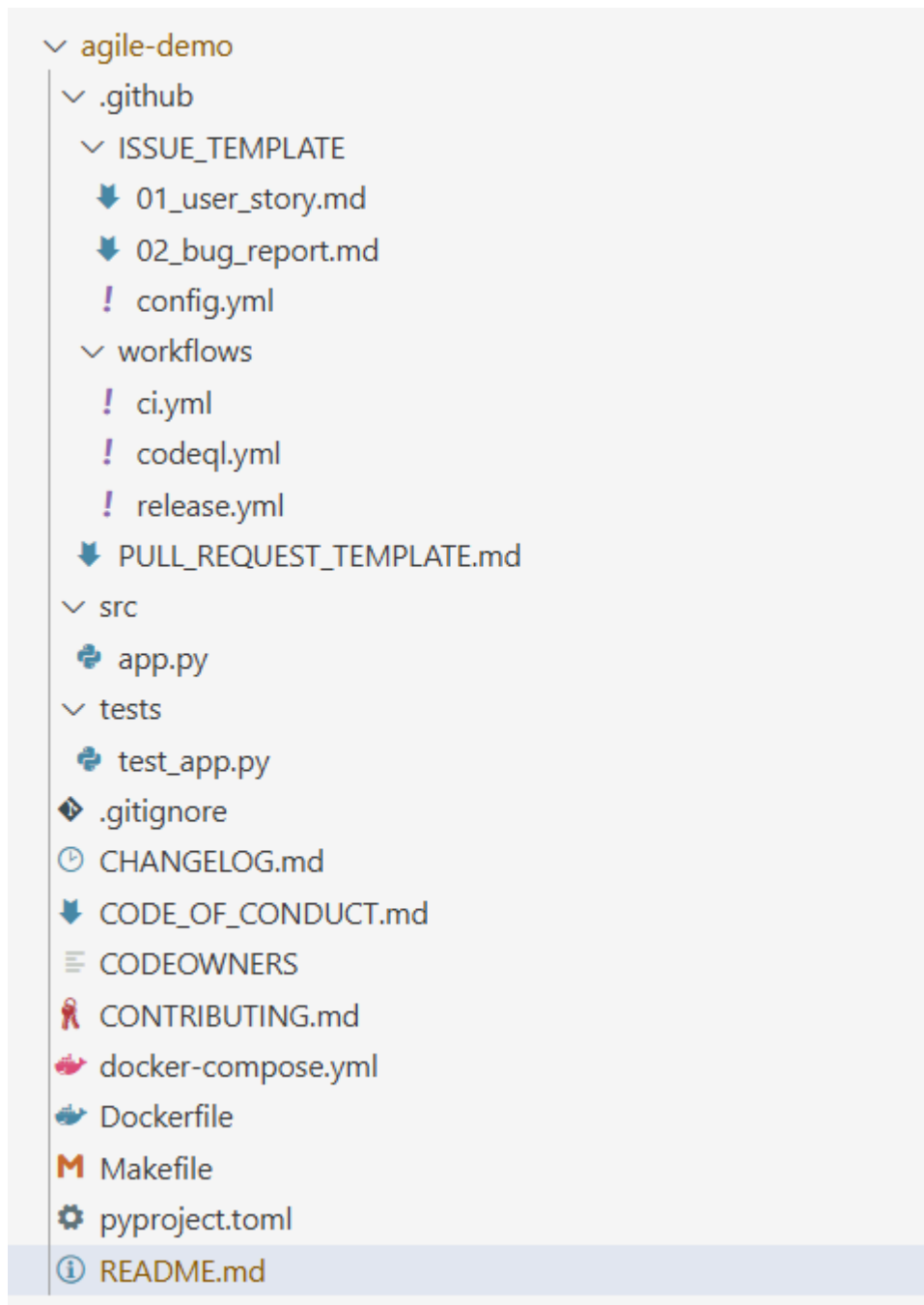
```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

7. Create Project Files

You can either **copy the template files** provided by your instructor or create them manually as shown in the class.

Your folder should look like this:



8. Install Dependencies

Run:

```
pip install -e ".[dev]"
```

This command installs:

- FastAPI (web framework)
- Pytest (testing)

- Black, Ruff, Mypy (code quality tools)

If pip is not detected, run below commands:

```
sudo apt update
```

```
sudo apt install -y python3 python3-venv python3-pip git make build-essential
```

9. Run Locally

To test your project:

```
python3 -m uvicorn src.app:app --reload
```

Then open your browser at:

<http://localhost:8000/health>

You should see:

```
{"status": "ok"}
```

Press **Ctrl + C** to stop.

10. Check and Fix Code Style

Before you push code to GitHub, always format and lint:

```
black .
```

```
ruff check .
```

black will automatically fix formatting.

If you see “All done!”, you’re ready to commit.

11. Save and Commit Your Work

```
git add .
```

```
git commit -m "Initial agile demo commit"
```

12. Create a GitHub Repository

1. Go to <https://github.com/new>
2. Name it **Agile-Demo**
3. Keep it **Public**

4. Don't tick any checkboxes (no README)
5. Click **Create repository**

Copy the HTTPS link GitHub gives you — it looks like:

`https://github.com/<yourname>/Agile-Demo.git`

13. Connect Local Folder to GitHub

Back in VS Code terminal:

```
git remote add origin https://github.com/<yourname>/Agile-Demo.git
```

```
git branch -M main
```

```
git push -u origin main
```

If it asks for authentication, allow it to open your browser and approve access.

14. Watch CI/CD Run on GitHub

1. Go to your GitHub repository online.
2. Click **Actions** tab.
3. You'll see your workflow "CI" running automatically.

It checks:

- Formatting (Black/Ruff)
- Tests (Pytest)
- Type checks (Mypy)

When everything is  green — you passed the Definition of Done!

15. Fixing Lint Errors

If CI fails and shows messages like:

```
would reformat src/app.py
```

That means your files need formatting.

Just run locally:

```
black .
```

```
git add .
```

```
git commit -m "style: format code"
```

```
git push
```

CI will pass next time.

16. Add New Features (Agile Style)

1. Open the **Issues** tab on GitHub.
2. Click **New Issue** → **User Story**.
3. Write your story in this format:

As a student, I want to list all items, So that I can see what's in my to-do list.

4. Add Acceptance Criteria:

Given I have items,

When I GET /items,

Then I see a list of items (status 200).

Then implement the story in your code and tests.

17. Branching and Pull Requests

When working in teams:

```
git checkout -b feat/list-items
```

```
# (make your changes)
```

```
git add .
```

```
git commit -m "feat: add list items endpoint"
```

```
git push origin feat/list-items
```

Then open a Pull Request (PR) on GitHub and wait for CI to pass and for review.

18. Release a Version

When your sprint ends and CI is green:

```
git tag v0.1.0
```

```
git push origin v0.1.0
```

Your workflow will automatically:

- Build your app into a Docker image
 - Publish it to GitHub Container Registry (GHCR)
-

19. Run Your Released Version

Once released:

```
docker pull ghcr.io/<yourname>/agile-demo:v0.1.0
```

```
docker run -p 8000:8000 ghcr.io/<yourname>/agile-demo:v0.1.0
```

Then check:

<http://localhost:8000/health>

20. What You Learned

- ✓ How to set up VS Code + WSL
 - ✓ How to use Git and GitHub
 - ✓ How CI/CD automatically tests your code
 - ✓ How Agile stories connect to real code
 - ✓ How to work like a modern software team
-

Congratulations — you've just built and deployed your first Agile CI/CD project!
Keep exploring and try adding your own new stories or improving test coverage.